SOFTWARE TESTING PRACTICAL REPORT

PRACTICAL REVIEW — PR-13

Student Name	Bhavesh Patil
Enrollment Number	2504030100419
Programme	BCA — Semester 3
Practical	PR-13 — Selenium Environment Setup / WebDriver Test
Tools	Python, Selenium WebDriver, Chrome, PyCharm

Submitted for academic practical evaluation

Prepared using the supplied execution screenshots. The report distinguishes the Tutorial 13 script from the Tutorial 12 output visible in the Run window.

1. Aim

To launch Google Chrome using Selenium WebDriver, open the Selenium website, print the page title and current URL, and keep the browser open for 60 seconds before closing it.

2. Objectives

• Verify that Python and Selenium are available in the PyCharm project environment.

• Start Chrome through Selenium WebDriver and maximize the browser window.

• Navigate to https://www.selenium.dev/ and read page information.

• Keep the browser visible for 60 seconds to allow observation and screenshots.

• Close the browser cleanly after the wait completes.

3. Software and Requirements

Component	Purpose
Python	Runs the automation script.
Selenium	Provides browser automation APIs.
Chrome	Browser controlled by WebDriver.
PyCharm	IDE used to write and execute the script.

4. Procedure

• Open the SeleniumTesting project in PyCharm and create tutorial13.py.

• Install Selenium in the same interpreter used by the project: python -m pip install selenium.

• Run the script using the project interpreter.

• Observe Chrome opening the target URL and review the printed title and URL in the Run console.

• The script waits for 60 seconds and then closes Chrome.

5. Program Code

```
from selenium import webdriver
import time

driver = webdriver.Chrome()
driver.maximize_window()
driver.get("https://www.selenium.dev/")

print("Browser launched successfully")
print("Page Title:", driver.title)
print("Current URL:", driver.current_url)

time.sleep(60)
driver.quit()
```

6. Code Explanation

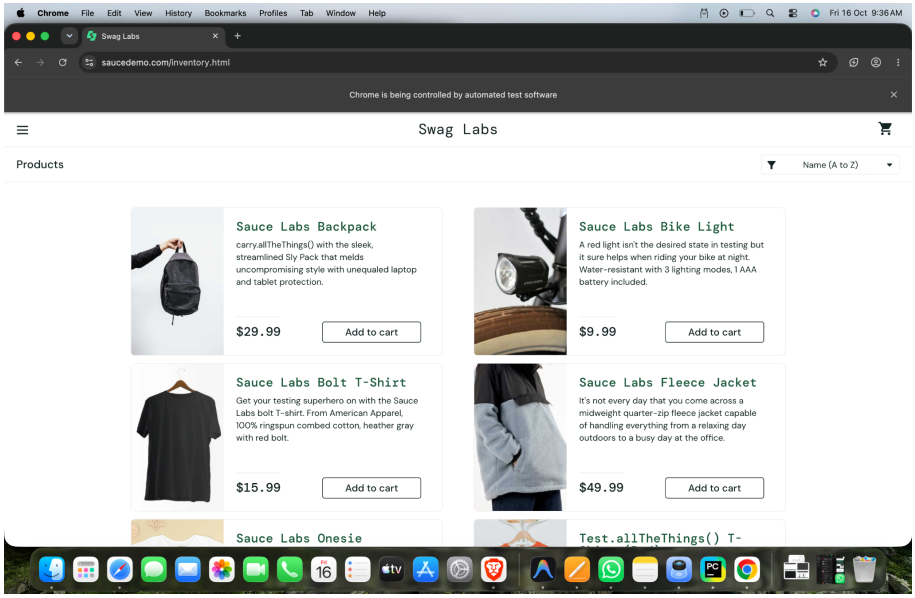
webdriver import	Imports Selenium's browser automation interface.
import time	Provides the sleep function used for the 60-second pause.

webdriver.Chrome()	Starts a Chrome browser session.
maximize_window()	Maximizes the browser window.
driver.get(...)	Navigates to the Selenium website.
print(...)	Displays a confirmation, the page title, and the current URL.
time.sleep(60)	Keeps the browser open for 60 seconds.
driver.quit()	Closes the browser session after the wait.

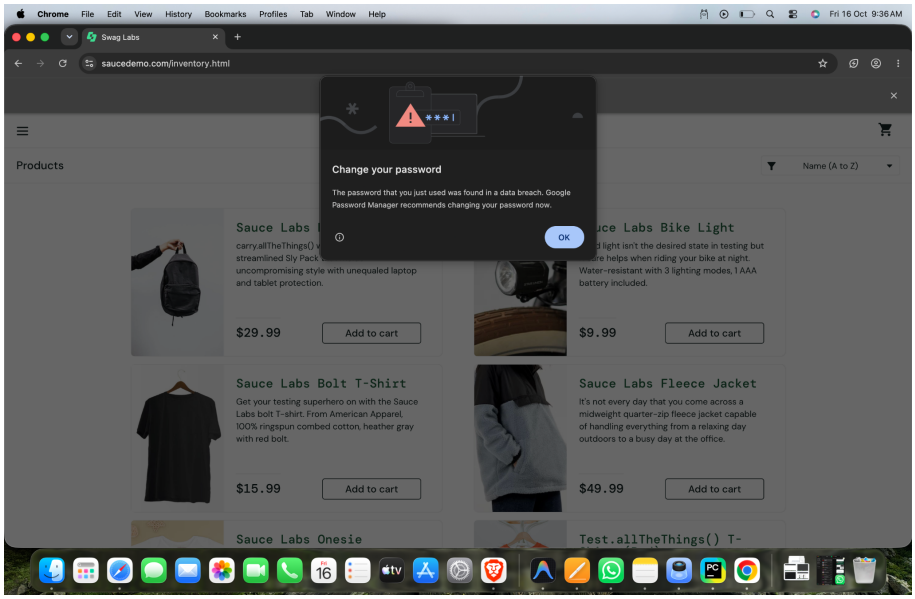
7. Evidence Screenshots

The screenshots below document the visible browser state, the security warning shown by Chrome, the Tutorial 13 source code, and the Run console. Note: the visible Run console shows Tutorial 12 (st12.py), not the execution output for Tutorial 13; therefore, it is not treated as proof that PR-13 passed.

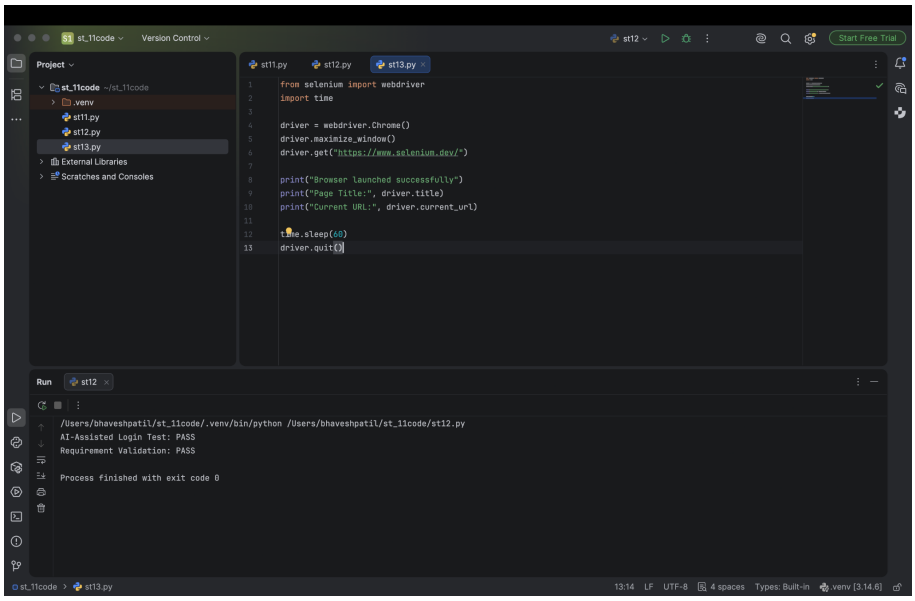
01 SauceDemo inventory page — products displayed after login



02 Chrome Password Manager security warning overlay



03 PyCharm — Tutorial 13 Selenium WebDriver script with 60-second wait



04 PyCharm Run window — previous Tutorial 12 test output (PASS)



8. Test Case Summary

Test ID	Test activity	Expected result	Evidence / status
TC-01	Launch Chrome	Chrome opens without WebDriver error.	Browser screenshot available
TC-02	Navigate to Selenium site	Selenium homepage is displayed.	Browser screenshot is available, but shows SauceDemo; UR
TC-03	Print title and URL	Title and URL are printed in Run console.	Tutorial 13 console output not supplied
TC-04	Wait 60 seconds	Browser remains open for 60 seconds.	Code screenshot shows time.sleep(60)
TC-05	Close browser	Browser session closes after wait.	Code screenshot shows driver.quit()

9. Observations

• The Tutorial 13 source code includes a 60-second delay before driver.quit().

• The browser screenshots show a SauceDemo inventory page, while the supplied Tutorial 13 code targets selenium.dev; these appear to be different test runs.

• Chrome displays a password-breach warning. This is a browser security notification, not a Selenium test result. Do not reuse a compromised password.

• The visible Run output reports PASS for an AI-assisted login test and requirement validation under st12.py; it belongs to Tutorial 12.

10. Result

The Tutorial 13 script is prepared to launch Chrome, visit the Selenium website, print page details, wait 60 seconds, and close the browser. The screenshots show the code and browser states, but a Tutorial 13 Run-console screenshot is still needed to confirm successful execution.

11. Conclusion

Selenium WebDriver can automate Chrome from Python. The 60-second wait allows visual verification before the browser session is closed.

Student: Bhavesh Patil | Enrollment: 2504030100419 | BCA Semester 3 | PR-13